



Matrixdock Gold XAUm

Security Assessment

May 15th, 2026 — Prepared by OtterSec

Renato Eugenio Maria Marziano

renato@osec.io

Andreas Mantzoutas

andreas@osec.io

Table of Contents

Executive Summary	2
Overview	2
Key Findings	2
Scope	3
Findings	4
Vulnerabilities	5
General Findings	6
OS-MGX-SUG-00 Inconsistent Timelock Boundary Across Chains	7
OS-MGX-SUG-01 Pool Account Address Type Mismatch	8
OS-MGX-SUG-02 Short-Lived Pending State Utilizes Instance Storage	9
OS-MGX-SUG-03 Mint Request TTL Not Maintained	10
OS-MGX-SUG-04 Code Maturity	11
Appendices	
Vulnerability Rating Scale	12
Procedure	13

01 — Executive Summary

Overview

Matrixdock engaged OtterSec to assess the `xaum-stellar` program. This assessment was conducted between May 4th and May 8th, 2026. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 5 findings throughout this audit engagement.

In particular, we highlighted an inconsistency in the enforcement of the timelock boundary across chains: EVM requires the current timestamp to be strictly greater than the effective time, whereas Stellar allows execution at the effective time ([OS-MGX-SUG-00](#)). Furthermore, PoolAccountA and PoolAccountB are documented and tested as MuxedAddress, but the setters accept plain Address type, resulting in a type mismatch and test failure ([OS-MGX-SUG-01](#)).

We also suggested extending the mint request TTL and storing mint requests in temporary storage ([OS-MGX-SUG-03](#)), and advised moving pending and timelock state to temporary storage to reduce persistent storage utilization and avoid incurring long-term rent costs for obsolete values ([OS-MGX-SUG-02](#)). Lastly, we advised improving functionality by requiring the new value to differ from the currently stored pending value when requesting an upgrade or ownership transfer, and by adding the ability to revoke an ownership transfer ([OS-MGX-SUG-04](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/Matrixdock-RWA/RWA-Contracts>. This audit was performed against commit [1b36bae](#).

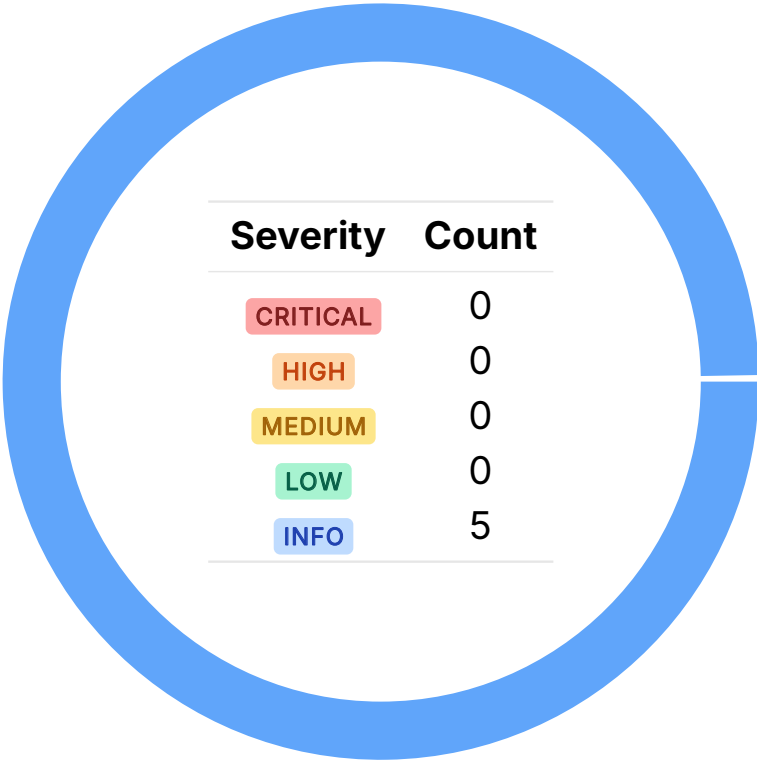
A brief description of the program is as follows:

Name	Description
xaum-stellar	Soroban implementation of a centrally managed XAUm token where the operator controls issuance/redemption flows and the owner controls governance and upgrade authority.

03 — Findings

Overall, we reported 5 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
----	----------	--------	-------------

05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MGX-SUG-00	Currently, EVM requires the current timestamp to be strictly greater than <code>et</code> , while Stellar allows execution when <code>now == et</code> .
OS-MGX-SUG-01	<code>PoolAccountA</code> and <code>PoolAccountB</code> are documented and tested as <code>MuxedAddress</code> , but the setters accept plain <code>Address</code> , resulting in a type mismatch and test failure.
OS-MGX-SUG-02	<code>et_*</code> and <code>pending</code> / <code>next</code> values are short-lived but stored in instance storage, increasing rent and invocation costs.
OS-MGX-SUG-03	Mint requests are stored persistently, but their TTL is not extended, so a pending request may expire before execution.
OS-MGX-SUG-04	Certain parts of the codebase can be updated to maintain consistency and ensure alignment with coding best practices.

Inconsistent Timelock Boundary Across Chains

OS-MGX-SUG-00

Description

There is a cross-chain semantic mismatch in how the operator-change timelock is considered executable. In the EVM implementation, execution requires `et < block.timestamp`, implying the action is only valid strictly after the effective timestamp. In the Stellar/Soroban implementation, the check only rejects `et > now`, so execution is allowed when `et == now`. This weakens consistency across XAUm deployment.

```
>_ xaum/src/contract.rs
```

RUST

```
pub fn set_operator(env: Env, new_operator: Address) {  
    [...]  
    if et != 0 {  
        // next_operator always be Some if et != 0, no need to check  
        if next_operator.unwrap() != new_operator {  
            panic_with_error!(&env, TokenError::PendingRequestExists);  
        }  
        if et > now {  
            panic_with_error!(&env, TokenError::TooEarlyToExecute);  
        }  
        state::write_operator(&env, &new_operator);  
        state::write_et_next_operator(&env, 0);  
        SetOperatorEffectuated {  
            operator: new_operator,  
        }  
        .publish(&env);  
        return;  
    }  
    [...]  
}
```

Remediation

Ensure alignment of implementations across chains to maintain consistency.

Patch

Fixed in [18b0573](#).

Pool Account Address Type Mismatch

OS-MGX-SUG-01

Description

The minter storage schema comments indicate `PoolAccountA` and `PoolAccountB` are intended to store `MuxedAddress` values, but the setter functions accept plain `Address` values. This creates a type/interface mismatch as callers and tests construct a `MuxedAddress`, while the contract entrypoint expects an `Address`, so the test calls to `set_pool_account_a(&new_pool)` and `set_pool_account_b(&new_pool)` do not match the function signatures.

```
>_ xaum-stellar/contracts/minter/src/storage_types.rs
```

RUST

```
#[contracttype]
pub enum DataKey {
    [...]
    PoolAccountA, // MuxedAddress
    PoolAccountB, // MuxedAddress
    NewWasmHash,  //BytesN<32>, hash of the new wasm to be set by upgrade
    [...]
}
```

Remediation

Update `set_pool_account_a` and `set_pool_account_b` to accept and store `MuxedAddress`, or update the storage comments, getters, and tests to consistently utilize plain `Address`.

Patch

Fixed in [9f66e5b](#), by updating the documentation and test suite to correctly indicate that the accepted type is `Address` instead of `MuxedAddress`.

Short-Lived Pending State Utilizes Instance Storage

OS-MGX-SUG-02

Description

The contract currently stores many short-lived `et_*` and `next / pending` values in instance storage, even though they are only required until a scheduled action is executed or revoked. Keeping these stale or temporary values in instance storage increases rent/extension costs and renders invocations more expensive because instance storage is loaded and extended as part of contract execution. Since these values only need to survive until the relevant timelock expires, temporary storage may safely cover the intended lifecycle.

Remediation

Move pending and timelock state to temporary storage to reduce persistent storage utilization and avoid incurring long-term rent costs for obsolete values.

Patch

Fixed in [91ff076](#).

Mint Request TTL Not Maintained

OS-MGX-SUG-03

Description

Mint requests are stored in persistent storage via `DataKey::MintRequest`, but unlike balances, `read_mint_request` does not extend the entry's TTL when accessed. Thus, a pending mint request may theoretically expire before execution if its storage lifetime is not maintained elsewhere, especially for longer delays or low-activity requests.

Remediation

Extend the mint request TTL on read/write, similar to `read_balance`, to ensure scheduled mints remain available until their effective time. Additionally, since mint requests are short-lived and removed once executed or revoked, temporary storage may be more appropriate persistent storage.

Patch

Fixed in [f717f4b](#).

Code Maturity

OS-MGX-SUG-04

Description

1. `request_upgrade` and `request_owner_transfer` do not check whether a pending upgrade hash, or pending owner, already exists before writing the new pending state. As a result, calling either function again overwrites the existing pending value and recomputes `effective_time`, effectively restarting the timelock. If the caller submits the same pending value again, this only delays execution without changing the intended target. Reject duplicate requests by requiring the new value to differ from the currently stored pending value.
2. The current implementation supports requesting and accepting an ownership transfer, but it does not provide a way to revoke a pending ownership transfer before acceptance. This implies that once `pending_owner` and `et_next_owner` are set, the owner cannot explicitly cancel the request if it was made inadvertently, or if the target owner becomes undesirable. Add functionality to revoke ownership transfer. This will also ensure ownership transfer handling is consistent with the existing revoke flows.
3. Most of the exported getters default to 0 if the relevant values do not exist in storage. While 0 has a special semantic meaning, off-chain consumers may treat it as a normal value and rely on it for off-chain decisions, such as interpreting an unset `et` as an expired timelock that is ready to be called. Update the getters to return `None` when the underlying value does not exist in storage, so that off-chain consumers can distinguish between an unset value and a meaningful 0.

Remediation

Incorporate the above suggestions into the codebase.

Patch

1. Fixed in [139eed6](#).
2. Fixed in [8a7f922](#).
3. Fixed in [6c5dc50](#).

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.